Florida Institute of Technology
Scholarship Repository @ Florida Tech

Link Foundation Modeling, Simulation and Training Fellowship Reports

Link Foundation Fellowship Reports

1999

Vortex Generated Rotor Wake Model for Real-Time Simulation

Kenneth S. Graham

Follow this and additional works at: https://repository.fit.edu/link_modeling

 Part of the Aerodynamics and Fluid Mechanics Commons

1999

Vortex Generated Rotor Wake Model for Real-Time Simulation

Kenneth S. Graham, Jr. and Amnon Katz

The Flight Dynamics Laboratory

University of Alabama

Box 870280

Tuscaloosa, AL 35487-0280

KGraham@hermes.fdl.eng.ua.edu

Akatz@hermes.fdl.eng.ua.edu

1. ROTOR MODELING AT THE UNIVERSITY OF ALABAMA

The University of Alabama Flight Dynamics Laboratory (UA FDL) has been simulating helicopters since 1981. The real-time, man-in-the-loop, simulator was developed locally and is continuously being upgraded. The hardware, software, architecture and math models are all developed in-house by a team of faculty and graduate students.

Since 1994, the UA rotor model has been *Bladehelo* [2] — a true blade model that numerically integrates the flapping motion of individual blades and the shaft rotation. The approach is physically based. Discrepancies are studied until they are understood. *Bladehelo* played a key role in the discovery of *residual bending* [3], [4].

Bladehelo uses rigorous dynamic equations for blade motion. There are no linearizations and no small angle approximations. The exact non-linear equations are integrated numerically. Current day computers, even modest ones, are equal to the task, and any labor spent on simplifying the equations is unnecessary and runs the risk of missing higher order effects like the one pointed out in reference [3].

Until recently, *Bladehelo* was subject to two major limitations: 1. rigid blades and 2. uniform inflow. Both of these limitations have been removed. The present paper reports on the modeling of a dynamic, vortex-induced wake.

2 UNSTEADY AERODYNAMICS (DYNAMICALLY INDUCED WAKE)

Every airborne vehicle, by resting its weight on air, causes a downwash. The vehicle flies through its own downwash and is, in effect, subject to half the downwash velocity. A down-flow is detrimental to performance and requires additional power to overcome. This is known as induced power (also related to induced drag). Induced drag and power are more significant, the slower the aircraft. Since helicopters fly down to zero airspeed, induced drag and power are most significant for a helicopter rotor.

The pre-existing *Bladehelo* models the downwash as a uniform inflow, whose magnitude is determined by conservation of momentum ("momentum theory"). This method of modeling assures a correct value for the average inflow and captures the essence of the physical effect during steady flight. When maneuvering, the uniform inflow model responds too quickly and too uniformly. It was pointed out by Prouty [9] that virtually all simulators compute the cross-coupling in transient maneuvers incorrectly. Rosen and Isser [7],[8] suggested that the actual observed effect is caused by distortion of the wake in transient maneuvers. These authors suggested that a wake modeled as a vortex lattice would predict transients correctly and reported that they had found it to be so.

In the current project, the ideas of [7],[8] were adapted to real-time simulation. In our version, the vortex lattice is related to a lifting line, rather than lifting surface, model of the blade elements. The lift generated by each blade element is still determined from local airspeed and angle of attack, using tables of lift coefficients. A trailing vortex is generated at the two ends of each blade element. A shed vortex is generated every integration step, when the lift on the blade element is recomputed. A model along these lines was created, coded and exercised off-line. In real time, limitations of computational throughput dictated that only tip and hub vortices be kept and that vortices be shed at a rate lower than the computational frame rate.

This reduced version of the dynamic wake was successfully run in real time, first interacting with the traditional *Bladehelo,* and then interacting with a structured modeling (SM) [5] version representing flexible blades. Time available did not allow an assessment of the effect of modeling upgrades on the fidelity with which handling qualities were represented.

3 ARCHITECTURE AND INTEGRATION.

The pre-existing *Bladehelo* ran on a single Pentium Pro processor as 16-bit code. Typically, the blade motion was computed at 330 frames per second, using Runge-Kutta integration. This provided more than adequate precision. It was clear, however, that the introduction the dynamic wake would make significant additional demands on computational resources.

It was expected that a transition to 32-bit computing would speed the computations up. Certain benchmarks suggested that a speedup by a factor of almost three could be achieved. These benchmarks were unconservative in that they did not allow for the communications of the simulation code with visual displays and cockpit controls. It was exactly the issue of communications that was holding up the conversion. The UA FDL had previously developed its own super-efficient interrupt driven communications schemes aimed specifically at real-time

simulation. These could not be accommodated in the newer operating systems that supported 32-bit computing.

When 32-bit computing was accomplished, it was found that the bottleneck in realizing a speed-up was actually in the 32-bit compilers that were available. The favorable benchmarks had been produced with the Intel optimized compiler that had been packaged with Borland C++ 5.01. That compiler proved too sensitive and unstable for the *Bladehelo* development work. The regular Borland compilers provided a speedup of only 25% to 35%. Eventually, the Code-Warrior compiler and an Intel compiler compatible with Microsoft C++ made it possible to realize a speed-up by a factor of nearly two.

Even, so, it was clear from the beginning that additional computing hardware would be required. After a survey of available systems, we selected four server-type computers running four Pentium II processors each. These four computers, purchased with FAA funds, were named *Monster0, Monster1, Monster2* and *Monster3*. The four "monsters" were interconnected by SCRAMNet (also purchased with FAA funds). The SCRAMNet also connected the four "monsters" to *Janus*, a two-processor computer located by the simulator cab and serving as simulation host. The "monsters" provide a nominal increase in computational power by a factor of 16 (or even 18 counting the two processors in *Janus*. These estimates are, of course, unconservative, since they make no allowance for the overhead involved in multiprocessing.

The dynamic wake required the adaptation of *Bladehelo* and wake code to multiprocessing and the creation of the required interfaces. At the present writing, multiprocessing has been achieved on a single "monster" (and on *Janus*) by multithreading. A SCRAMNet interface *Bladehelo* running on one computer and the dynamic wake running on another has also been achieved. These accomplishments were sufficient to achieve the running of *Bladehelo* with the dynamic wake in real time. Multiprocessing using all 16 (or 18) processors together is still under development.

Appendix A of this report covers the architectural and interface issues addressed above.

4 WAKE MODEL DESCRIPTION.

The preexisting *Bladehelo* modeled the wake as a uniform inflow based on "momentum theory." The total aerodynamic force created by the rotor was applied to an appropriate mass flow, resulting in a downwash. The rotor was subjected to half of its own downwash.

The new model developed under this project follows the ideas of references 7 and 8, and derives the wake flow from bound, shed, and trailing vortices generated by the rotor. Unlike the methods described in the references, the vortices are derived using the lifting line rather than the lifting surface scheme. *Bladehelo* models each rotor blade as a sequence of finite blade elements. The aerodynamic force generated by each rotor blade is computed by summing the aerodynamic force generated by the blade elements. The aerodynamic force of an element is determined using the local airspeed at the element (including induced flow) with lookup tables for the lift and drag coefficients. The wake model provides, to the rotor model, the induced velocity of the air at each blade element's location.

As a rotor blade moves through the air, vorticity is shed in a continuous sheet from along the trailing edge of each rotor blade. In *Bladehelo*, both the physical blade and its motion have been made discrete. The lift is computed for the elements and assumed uniformly distributed along them. These values of lift per unit span are computed only at discrete time steps. The discrete representation of span and of time dictates an approximation of the vortex sheet as a discrete vortex lattice. The vortex lattice forms the wake and moves away from the rotor with the air. Each lattice forms an essentially helical shape with the sheets intertwined. The wake model keeps track of the positions of the vortex lattices over time.

Each blade element is treated as a lifting line. The strength of a bound vortex segment is computed using the Kutta-Joukowski theorem

$$\Gamma = \frac{L'}{\rho V}, \tag{1}$$

where L' is the lift per unit span for the element, ρ is the air density, and V is the magnitude of the component of velocity of the air relative to the element that is normal to the lifting line.

A trailing vortex is generated at the blade element boundaries and carries the difference of bound vorticity (circulation) about the two elements. Vortices are shed from each blade element only once a step and carry the difference between the circulation about the element from the last step to the current step. Trailing vortices from the tip and root of the blade carry the full circulation of these elements. These are the strongest trailing vortices. In real time, they are the only ones considered. We refer to the trailing vortices of each element as "chord-wise segments" and to the shed vortices as "span-wise segments." The points where lattice segments meet are called nodes.

A free vortex filament maintains a constant strength and remains stationary relative to the air, while the blade element moves away In accordance with the vortex theorems of Helmholtz, vorticity is conserved. The vortex filaments cannot end in the fluid. Vorticity is conserved through each node of the lattice. The wake simulation keeps track of the position of each node over time. The vortex segments connecting the nodes are assumed to be straight. The initial positions of the nodes are the positions of the end points of each blade element of the rotor model. After creation at the initial positions, the nodes move at the same velocity as the air. Therefore, by determining the air velocity at each node, the position of each node can be propagated forward in time by numerical integration. Figure 1 illustrates a part of a vortex lattice from one blade.

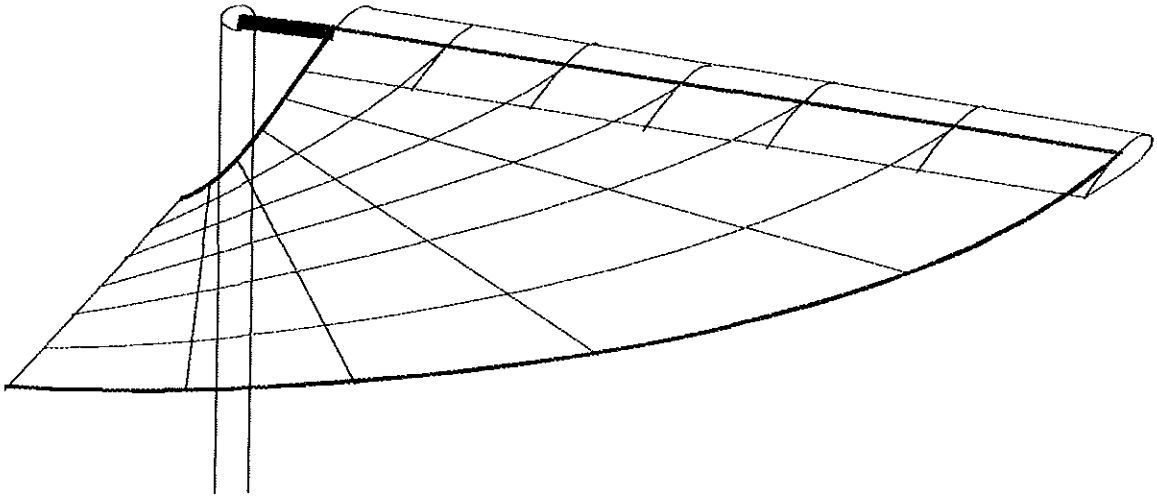


Figure 1: Part of the vortex lattice from a single blade.

The vortex lattices are used to determine the velocity of the air at the center of the aerodynamic elements for use in computing the lift and drag. The lattices are also employed to determine the air velocity at each of the lattice nodes so that the positions of the nodes can be propagated forward in time. The Biot-Savart law can be used to determine the air velocity induced by a vortex filament at any point of interest.

$$\vec{v}_i = \int \frac{\Gamma(\vec{r} - \vec{p}) \times \overrightarrow{dr}}{4\pi |\vec{r} - \vec{p}|^3}, \tag{2}$$

where Γ is the strength of the filament, $\vec{r}$ is a radius vector to a point on the filament, $\vec{p}$ is the radius vector to the point of interest, and the integration is carried along the curved filament. Because the equations that govern the flow field are linear, the Biot-Savart law can be applied to each straight vortex segment, as if each segment induced a portion of the air velocity, and then the contributions from all of the individual segments can be summed to determine the total air velocity induced by all of the lattices.

5 COMPUTATIONAL DETAILS

5.1 VELOCITY INDUCED BY A VORTEX SEGMENT

When applied to a straight filament, equation (2) yields

$$\vec{v}_i = \frac{\Gamma \hat{s} \cdot (\hat{e}_1 - \hat{e}_2)}{4\pi h} \frac{\hat{e}_1 \times \hat{e}_2}{|\hat{e}_1 \times \hat{e}_2|}, \tag{3}$$

where h is the perpendicular distance between the segment and the point of interest, $\hat{e}_1$ and $\hat{e}_2$ are unit vectors pointing at the point from the two ends of the segment, and $\hat{s}$ is a unit vector along the segment.

Our model employs a less computationally intensive approximation to (2), namely

$$\vec{v}_i = \frac{\Gamma l \hat{s} \times \vec{h}}{4\pi \left|\vec{h}\right|^3}. \tag{4}$$

In equation (4), $\vec{h}$ is a vector from the midpoint of the segment to the position at which the induced velocity is being computed and l is the length of the segment. Substitution of (3) for (2) would be compatible with our model, because the vortex segments are straight. The use of the further simplified (4) is permissible because the position at which the induced velocity is being computed is usually a "long" distance from the segment, compared to the length of the segment. Also, the extra precision of (2) would be out of place in the context of the approximation of a discrete lattice.

5.2 SMOOTHING

In equation (4), as $\vec{h}$ approaches zero, the induced velocity becomes infinite. This is an unrealistic result that arises from the discretization of vortices. At a distance, the discrete segments approximate the same effects as the vortex sheets, but at close range the discrete segments introduce large ripples in the solution. The approximation of discrete vortices is not valid close to a vortex filament. Initial trials with the wake model using equation (4) resulted in instabilities in the solution because some points at which the induced velocity was computed were too near vortex segments. To compensate for this effect, a smoothing mechanism has been incorporated into the model. For computations of the contribution to the induced velocity for a segment, when the point of interest is closer to the segment than a given smoothing radius, R_s, equation (4) is replaced by

$$\vec{v}_i = \frac{\Gamma l \hat{s} \times \vec{h}}{4\pi R_s^3} \tag{5}$$

Equation (5) gives the average velocity induced by a segment over a sphere centered at the point of interest when the segment midpoint is within the sphere. For a rotor with an approximate radius of 24 feet, a smoothing radius of 4 feet has been found to give reasonable results. Investigation into the appropriate value for the smoothing radius or into the appropriate rotor parameter on which to base the smoothing radius will be conducted in the future.

5.3 LATTICE TRUNCATION

While the wake model is running, a new set of nodes and vortex segments – spanwise and chordwise – is added to each lattice at the rotor blade each wake simulation frame. Each such set is referred to as a generation. The generation closest to the blade is the youngest. Theoretically, the vortex segments continue to exist forever. This implies that as a simulation continues, the

computational requirements for keeping track of the wake grow indefinitely as more generations are added. The computer program can handle only a finite number of generations. Therefore, after a predefined number of generations have been included in the wake lattices, whenever a new generation is added at the blade, the oldest generation is discarded.

This procedure is a computational necessity. The magnitude of the velocity induced by the segment is inversely proportional to the square of the distance, and the oldest generation is usually the farthest from the rotor blades. Therefore, discarding the oldest generation should not greatly impact the accuracy of the induced velocity computations at the rotor blades, as long as a sufficient number of generations are retained.

6 THE NUMBER OF REVOLUTIONS OF WAKE THAT NEED BE RETAINED

To estimate the number of generations required, the ideal uniform induced velocity for hover has been compared to the velocity induced at the hub of the rotor by a number of vortex rings. For the ideal case of uniform disk loading in hover, the vortex strength along the rotor blades is constant. Equation (1) shows that this is the case when the lift per unit span is proportional to the velocity. The velocity is proportional to the radial position. Therefore, the lift per unit span must be proportional to the radial distance along the rotor blade.

Let B stand for the number of rotor blades. The non-dimensional thrust coefficient, C_T, can be computed in terms of Γ by integrating the lift per unit span of (1) along the rotor blades, each of length R, in the following manner:

$$C_T = \frac{B\int_0^R \rho\Gamma\Omega r dr}{\frac{1}{2}\rho\pi R^4\Omega^2} = \frac{B\Gamma}{\pi R^2\Omega}. \tag{6}$$

Therefore, the strength of the bound vortex on each blade is given by

$$\Gamma = \frac{C_T\pi R^2\Omega}{B} \tag{7}$$

For the ideal case of constant vortex strength along the blades, all of the vorticity is shed at the blade tips. Therefore, equation (7) also gives the strength of the free vortex filaments that trail from the rotor blade tips in hover. The tip vortices normally form intertwined helical patterns as they move away from the rotor. For this estimate, however, the tip vortices were modeled as rings. Each ring represents the trailing vortex produced over one revolution of the rotor and has a strength equal to the sum of the strengths of the vortices shed from all the blades, i.e. the strength of the rings is $B\Gamma$.

The radius of the rotor is used for the radius of each ring. In reality, the radii of the shed vortices contract and the air velocity increases as the tip vortices move away from the rotor, but for the current estimate of the conditions near the rotor, these effects will be ignored. The air velocity induced by a vortex ring that is a vertical distance h below the rotor is determined by integrating

equation (2) around the ring. At the hub, which is centered above the rings, the induced air velocity has only a vertical component, which comes out as

$$v_i = \int_0^{2\pi} \frac{B\Gamma R^2}{4\pi p^3} d\theta = \frac{B\Gamma R^2}{2p^3}, \tag{8}$$

where p is the distance from the ring to the hub as shown in figure 2. Equation (8) can be non-dimensionalized by dividing by the speed of the rotor blade tip, ΩR. After substituting equation (7) for Γ, the non-dimensional inflow parameter is given by

$$\lambda_i = \frac{\pi R^3 C_T}{2p^3}. \tag{9}$$

The vortex rings move away from the rotor at the speed of the air. Assuming the air velocity remains constant downstream from the rotor, the vertical distance from the hub to the nth ring is given by

$$h_n = v_i n \frac{2\pi}{\Omega} = \lambda_i R 2\pi n \tag{10}$$

Replacing p with

$$p_n = R\left(1 + (h_n/R)^2\right)^{1/2}, \tag{11}$$

equation (9) becomes

$$\lambda_{i_n} = \frac{\pi C_T}{2\left(1 + (h_n/R)^2\right)^{3/2}}. \tag{12}$$

The inflow parameter given by actuator disc theory for uniform inflow is

$$\lambda_i = \sqrt{C_T}/2 \tag{13}$$

Using the inflow parameter from equation (12), and dividing by the rotor radius to non-dimensionalize, equation (10) becomes

$$\frac{h_n}{R} = \pi n \sqrt{C_T}. \tag{14}$$

Substituting equation (14) into equation (12) and summing over N rings gives an estimate for the velocity induced at the rotor hub by the tip vortices generated over N rotor revolutions,

$$\lambda_i = \sum_{n=0}^{N-1} \frac{\pi C_T}{2\left(1+n^2\pi^2 C_T\right)^{3/2}}. \tag{15}$$

This equation is plotted versus N in figure 3. The solid line is the value for the inflow parameter given by equation (13) for ideal uniform inflow. The thrust coefficient used for both plots is for a UH-1H helicopter in hover ($C_T = 4.05\times10^{-3}$). The plot indicates that the uniform inflow value would be achieved with approximately 11 rings, with each ring representing one revolution's worth of the tip vortices.

Based on the above estimate, it was decided, as a rule of thumb, that the wake produced by ten rotor revolutions was to be retained. The inaccuracy introduced appears modest, and the very limited computational resources would hardly support more.

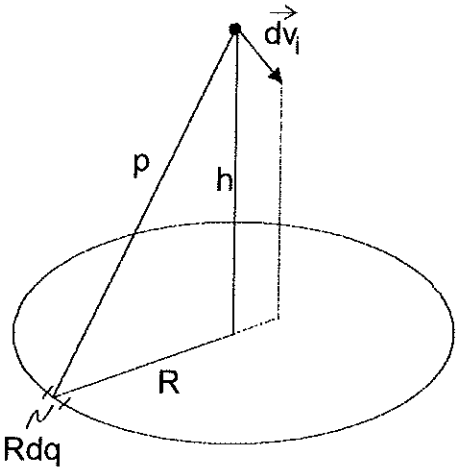


Figure 2: Velocity induced by a vortex ring.

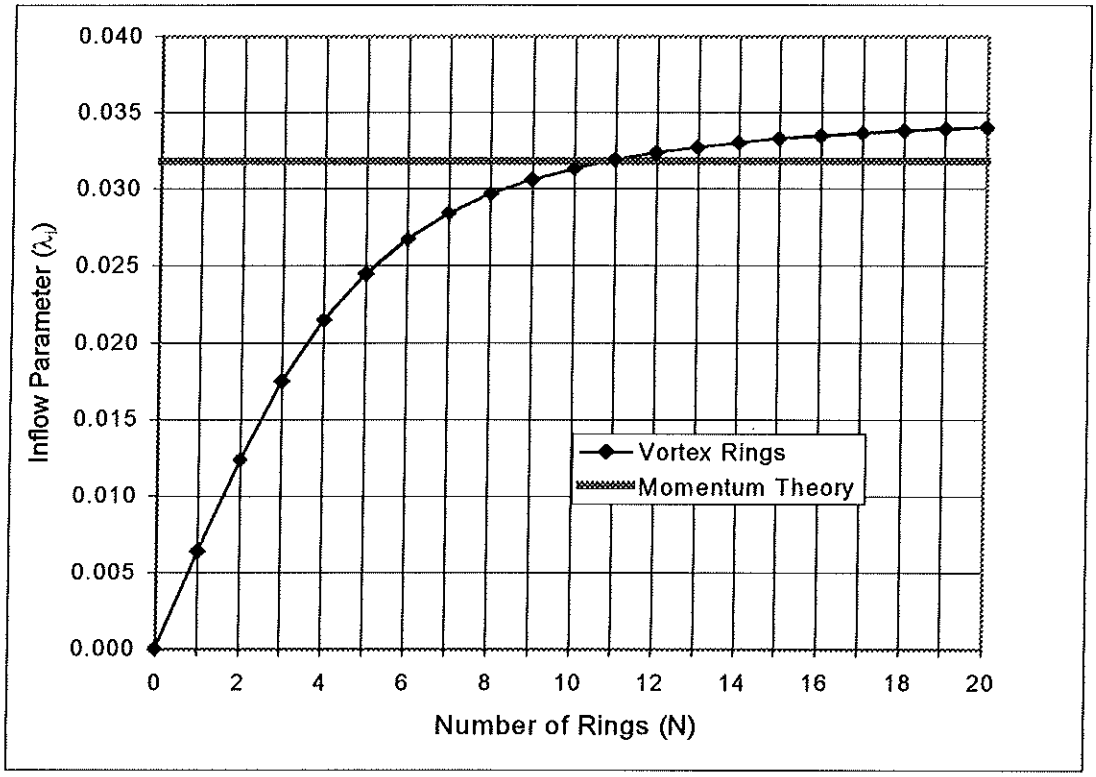


Figure 3: Number of revolutions required to reproduce induced velocity in hover.

7 COMPUTATIONAL REQUIREMENTS.

During simulation of the wake, repeated application of equation (4) consumes the majority of the computational resources. The time required for one use of equation (4) was approximated by computing the function for several seconds and dividing the number of computations by the time. It was found that for a computer with a 400 MHz Pentium II processor, the function that computes equation (4) takes 4.125×10^{-7} seconds. For one wake simulation frame, the number of times this function is called is given by

$$C = NS, \tag{16}$$

where N is the number of nodes and S is the number of vortex segments in the lattice. If n is the number of nodes per generation per blade, B is the number of rotor blades, and G is the number of generations retained, N is given by

$$N = BGn \tag{17}$$

The number of vortex segments is given by

$$S = B(2nG - G - n) \tag{18}$$

Substituting equations (17) and (18) into equation (16) gives

$$C = 2B^2G^2n^2 - B^2G^2n - B^2Gn^2 \tag{19}$$

Using typical *Bladehelo* parameters, we set $B = 2$ (two blades), and $n = 11$ (ten elements per blade). We assume eight wake updates every rotor revolution, which, for ten revolutions retained, gives $G = 80$. With these assumptions, equation (4) will be called 5,874,880 times. The time for one wake frame, just for equation (4), would be 2.42 seconds. For a UH-1H helicopter, 1/8 of a revolution takes only approximately 0.023 seconds. For this configuration, the wake calculations would take more than 100 times longer than real time.

To reduce the computational requirements and allow real-time use of the model, the number of nodes per generation had to be reduced to only two per rotor blade. In this mode, there is only one vortex segment along the blade instead of one segment per blade element. The strength of the vortex segment is taken to be the average of the strengths of the segments that would have been used for each aerodynamic element. Chord-wise segments are shed only at the blade tips and roots. This is believed to still represent a reasonable model because, in reality, the tip and root vortices are the strongest vortices and have the greatest effect on the wake.

For this choice, $C = 152{,}320$ and the time for one wake frame would be 0.063 seconds. This would still be almost three times real time. To overcome this obstacle to real-time use of the model, multiple CPU PC's are being employed. The computations of the induced velocity at each node can be performed in parallel.

Multiprocessing of the wake code with the four 400 MHz Pentium II processors residing on a common bus has been demonstrated. A factor of four applied to the estimate above should permit real-time computation. In reality, only a portion of wake representing six rotor revolutions could be retained in real time. Application of a factor of four to the estimate above would have been unconservative on several counts. No allowance was made for

a) Computations other than equation (4).

b) More complex housekeeping when equation (4) is employed in the wake.

c) Interface of the wake code with *Bladehelo* running on a different platform

As it stands, only 6 revolutions of wake could be retained in real time. Keeping ten revolutions required 133 seconds to represent 60 seconds of real time -- a factor of 2.2 behind. Note that on the basis of equation (19) the time required is roughly proportional to the square of G. With only 60% of the required nodes retained, only 36% of the calls to equation (4) were made. One would expect to be behind real time by a factor of 2.8. However, with the somewhat reduced wake, equation (4) no longer completely dominates the required resources. The factor of 2.2 was determined directly. It appears that the nominal factor of four available by the parallel use of four four-processor computers will overcome the 2.2 deficiency and bring us back to real time with ten revolutions retained.

8 MOMENTUM THEORY UNIFORM CORRECTION

Anticipating that the amount of wake that could be retained in real-time computations might not be sufficient to produce the correct downwash, the algorithm was modified to include a superimposed uniform inflow in the amount necessary to restore momentum balance. The uniform correction stands in for the effect of far vortices that have been discarded. The nearby vortices are traced specifically and any effect of their geometry on the local flow at the rotor blades is included in the model. The vortices discarded are too far for their detailed geometry to matter. A uniform inflow can adequately account for their contribution to the induced flow.

The uniform correction serves also a second purpose. When the program starts, initially there is no wake and no inflow. When the wake of the first few revolutions is generated, there is no downwash to move the vortices away from the rotor and to space them from each other. Without the uniform correction, the tip vortices would initially remain too close together, which could trigger instabilities and singularities. With the smoothing algorithm in force, the effect of each vortex segment on the others could be neutralized, preventing the wake from ever spreading out.

These difficulties are avoided by having the uniform correction in effect. This way, there is inflow in roughly the correct amount in place immediately, which helps float the trailing and shed vortices away and space them appropriately. As the wake builds up, the vortex lattice produces more and more inflow, and the uniform correction is reduced accordingly. Once the wake is grown to the extent planned, the correction is no longer applied to the lattice nodes. But the correction is still applied at the blades.

The uniform correction is based on the average of the inflow produced by the vortex lattice as computed at an appropriately chosen grid of points in the plane of the rotor disk. This average is determined continuously, and the correction is adjusted accordingly. Should the wake be able to account for the entire momentum balance, the uniform correction then would automatically reduce to insignificance.

Figures 4, 5, and 6 trace the history of the wake as it builds up and in the steady state that is reached when retaining 6, 10, or 20 revolutions of wake. These figures are for a UH1 in OGE hover with only tip and hub vortices retained and with eight steps per revolution. The modeled vortices thus enclose a hexagon whose area is 10% less than the disk area. The inflow is accordingly increased. The correction is based on momentum balance. It is seen that the correction converges to zero, while the modeled inflow becomes 10% greater than the reference uniform value.

Figures 4 through 6 were produced off line by employing the exact contribution of a segment according to equation (3). Figure 7 shows the same quantities as figure 4, but computed with the approximate equation (4) that can be supported in real time. In fact, the data of figure 7 (six revs retained) are commensurate with the real-time capabilities at this writing. It is seen that the data of figure 7 is a reasonable approximation to the more accurate figure 4. Figure 8 shows a ten-revolution vortex lattice in OGE hover that produced the data in figure 5.

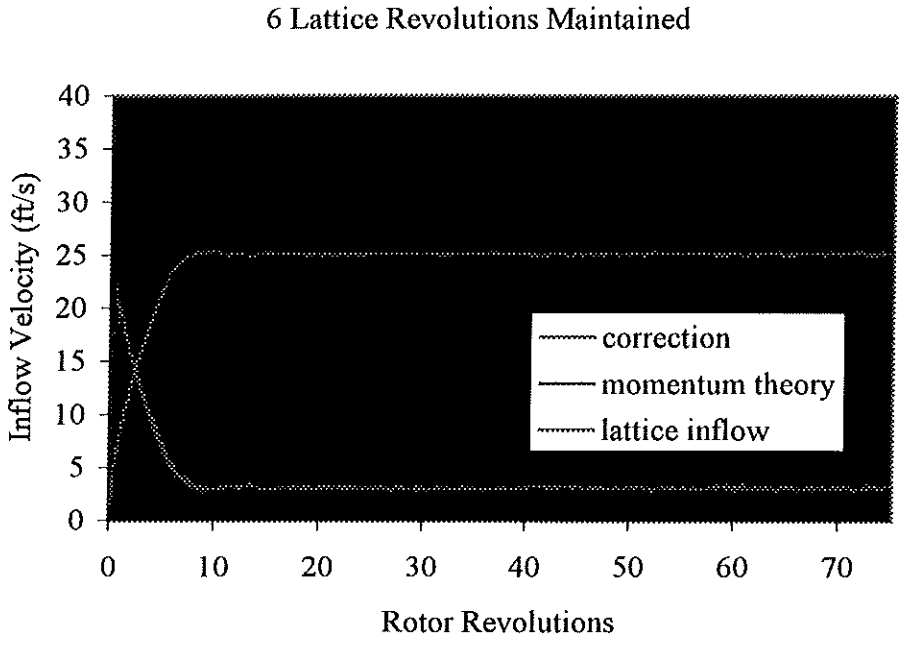


Figure 4: Inflow produced by wake model at six revolutions (eq . (3)).

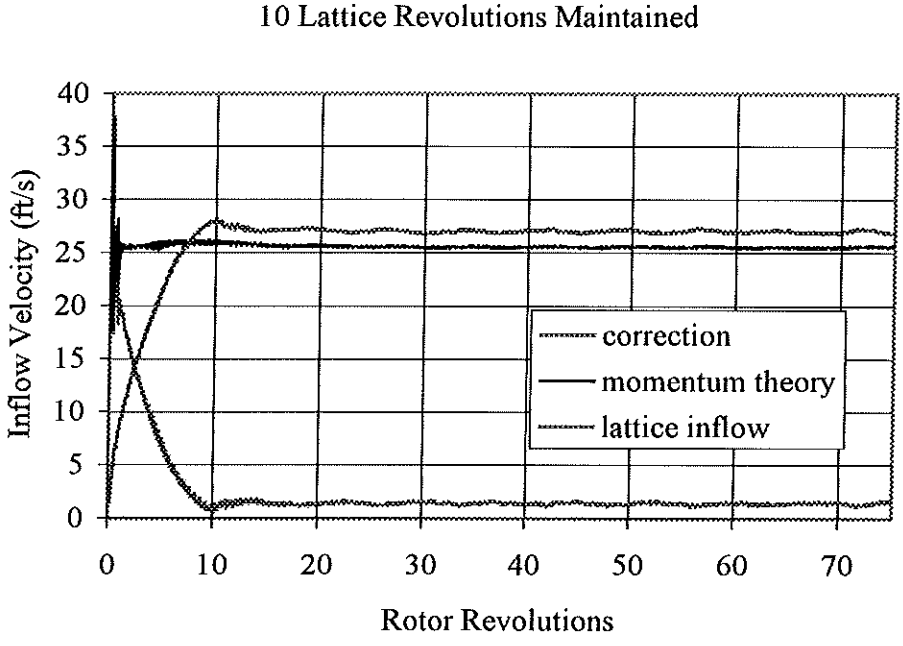


Figure 5: Inflow produced by wake model at ten revolutions (eq . (3)).

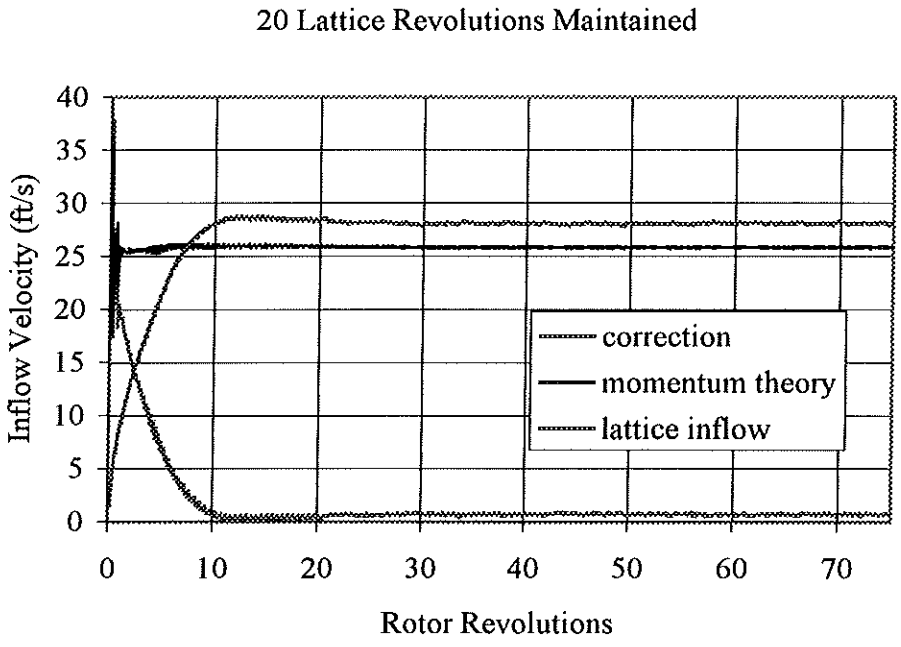


Figure 6: Inflow produced by wake model at twenty revolutions (eq . (3)).

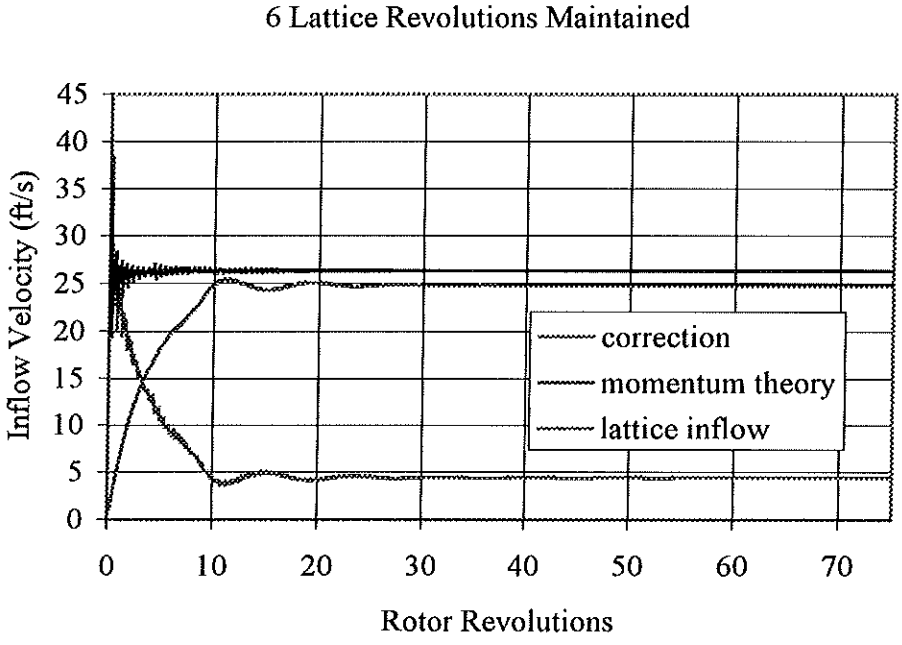


Figure 7: Inflow produced by wake model at six revolutions (eq . (4)).

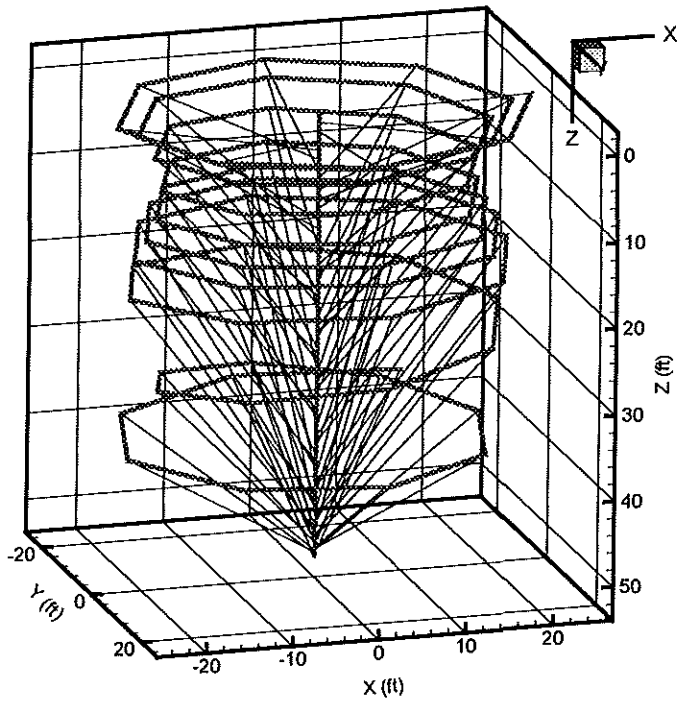


Figure 8: Ten-revolution wake in OGE hover.

9 MODEL SUMMARY.

The wake model is initialized with one set of spanwise vortex segments on each blade. The following algorithm is then employed each wake simulation frame:

- Add a new generation of nodes and vortex segments to the lattices at the rotor blades using equation (1) to compute the vortex strengths on the blades.
- Once the wake is "full", discard a generation each time a generation is added.
- Compute the induced velocity at the control point of each aerodynamic blade element and at each node of the lattice using equation (4) with each vortex segment in all of the lattices.
- Integrate to find the new inertial position of each node in the lattices using the previously computed induced velocities.
- Uniform correction automatically assessed and applied.

10 EFFECT OF WAKE MODEL ON FLIGHT CHARACTERISTICS

Figure 9 shows the response of *Bladehelo* to a forward cyclic step input, with and without the vortex lattice wake model. A history of orientation is plotted against time. The flight condition is OGE hover. The helicopter is held fixed for five seconds, during which time the wake builds up and stabilizes. The helicopter is the released while, at the same time, a forward cyclic input is applied. For comparison, a history without the step is also shown, as are histories with and without the step based on the preexisting uniform inflow model. In all cases, the control inputs prior to release are trimmed for hover with the uniform inflow model. It is seen that the simulation based on the new wake model is less stable and deviates from the trim condition faster. In either case, the longitudinal response to forward cyclic is pitch down. However the lateral response is dramatically different. With uniform inflow, the helicopter rolls right. With the dynamic wake, it rolls left.

The dynamic wake model was expected to improve the realism of the simulation in transient situations when the shape of the wake is distorted. Prouty [9] pointed out that simulations tend to predict the cross-coupling between longitudinal cyclic and lateral response incorrectly. In simulation, a forward step input results in roll to the right, while in reality it is to the left. The reference suggests that distortion of the wake, which is usually not modeled, is the reason. The results in figure 9 tend to corroborate this view, however, figure 9 is a very preliminary result that is included here as an illustration. A great deal of work is still pending to assess the effect and validity of the new wake model.

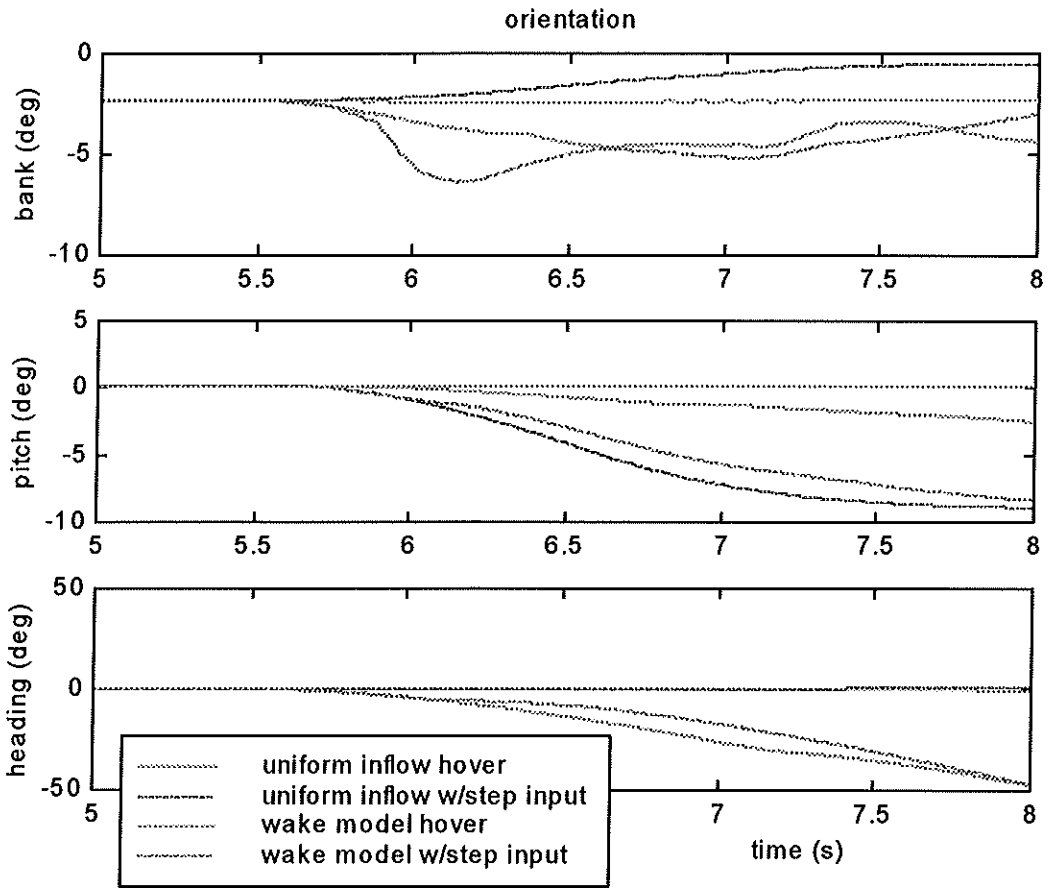


Figure 9: Comparison of control response with vortex lattice wake and with uniform inflow.

APPENDIX A: ARCHITECTURE

A.1 SIMULATION HOST HARDWARE UPGRADE.

At the start of the current project, in May 1998, the host computer at the UA FDL was named *Archimedes* -- a PC with a 200 MHz Pentium Pro microprocessor running MS DOS as the operating system. The host simulation code utilized the microprocessor in 16 bit mode. The simulation frame rate for the body equations was 85 Hz, and the frame rate for the rotor blade dynamics was greater than 300 Hz, using RK4 (fourth order Runge-Kutta) integration, which is the equivalent of 1200 Hz using the AB4 (Fourth Order Adams Bashforth) integration scheme. These rates provided excellent accuracy for the dynamics models that were then in use. However, it was clear that attaining similar frame rates while using the structured modeling based rotor simulation or the vortex lattice wake model would require much more extensive computational resources (see section A.3). For initial development, *Archimedes* was replaced by *Hamilton* -- a 400 MHz Pentium II based PC. This hardware upgrade doubled the performance. *Hamilton* has remained the simulation host for the DOS based versions of *Bladehelo.*

A.2 COMPILERS

A.2.1 BORLAND COMPILERS.

To increase the performance further, the simulation code was converted to be a Windows application so that it could utilize the processor in 32 bit mode. Access to hardware devices is accomplished in a much different fashion for the Win32 Application Programmer's Interface (API) than for DOS. Therefore, for the initial conversion, only the model and the main simulation loop were converted. The serial communications and the reading of the analog to digital converter - used for the flight controls- were not included. The code was compiled using an Intel created optimizing compiler bundled with the Borland C++ 5.01 compiler. Operating in this fashion, the 32 bit version of the simulation was approximately 2.8 times faster than the 16 bit version.

Next, the serial communications and the reading of the analog flight controls was added to the Win32 version. The speedup over the 16 bit version dropped to about 2.4 times faster. As the program became more complex, however, use of the Intel optimizing compiler became more difficult. Minor changes in the program would cause the compiler to crash unpredictably, or to flag previously accepted syntactically correct code as an error. Unexplained changes would be required in unrelated portions of the program to make compilation possible. The amount of time required to successfully compile the simulation after each minor change made the optimizing compiler impractical. In addition, the results obtained from the simulation compiled with the optimizing compiler sometimes varied slightly from the results obtained with the Borland C++ 5.01 compiler. This further degraded confidence in this particular Intel compiler. For these reasons, the Intel compiler was abandoned and the standard Borland C++ compilers, versions 5.01 and 5.02, were used. These compilers provided only about 30% speed improvement over the DOS version.

A.2.2 CODE WARRIOR

The most recent version of the Borland C++ compiler was several years old. An effort was made to identify a more modern compiler that could produce executables with better performance. The first one tried was the Metroworks CodeWarrior C++ 4.0 compiler. With optimizations turned on, the resulting executable ran at just under 70% faster than the 32 bit Borland version. The compiler was very stable, even with optimizations turned on. The C++ syntax supported very closely follows the ANSI C++ standard, making it more convenient for programming. Compilation times with optimization on, however, are very long. A full compile of the simulation host program could take up to 40 minutes. Use of the compiler for development work was more practical with optimizations turned off. In this mode, a complete compile takes 5-10 minutes. Compile times were kept down by only compiling with optimization for the final versions and through the use of project files that only rebuild the files where a source code change was made.

A.2.2 INTEL VTUNE ENVIRONMENT / MICROSOFT VISUAL C++.

An updated version of the Intel optimizing C++ compiler is available directly from Intel as part of their VTune Performance Enhancement Environment. It is designed to take maximum advantage of Intel microprocessors up to a Pentium III processor. As part of an educational institution support program, Intel donated a copy of the latest version, 4.0, of the VTune package to the UA FDL. The Intel compiler recognizes the same C++ syntax as Microsoft Visual C++ 6.0 and can be used as a plug-in for the Visual C++ developer's studio. Microsoft C++ has some deviations from the ANSI C++ standard which necessitated some modifications to the FDL source code before it could be compiled with either the Microsoft or the Intel compiler.

Compile time using the Intel compiler is significantly longer than for the Microsoft compiler, although it is shorter than for the Code Warrior package with optimizations on. Compilation of *Bladehelo* from scratch requires about 25 minutes. To avoid the long compile times during development, the Microsoft compiler was employed until the conversion of the code was complete. The executable resulting from the Microsoft compiler ran slower than the executable produced by the Borland C++ compiler. Once the incompatibilities were resolved, the Intel compiler was used. The resulting executable runs about 70% faster than the Borland compiled program, and is, therefore, about twice as fast as the 16 bit DOS version.

Because the compile times are significantly shorter, and the resulting program is slightly faster, the Intel compiler is currently being used instead of the Code Warrior compiler. The VTune package also includes performance analysis tools that can be used to improve computational efficiency. Use of these tools will be investigated in the future.

A.3 DISTRIBUTED HOST PROCESSING.

It was anticipated that, even with the added speed provided by 32 bit operation, the vortex lattice wake would require substantial added hardware resources to achieve real-time operation. Selecting and acquiring the new hardware, as well as installing and integrating it was an important part of the present project. A system of multiple networked computers, each with multiple CPUs, was selected. It consists of four computers that we named *Monster0, Monster1,*

Monster2 and *Monster3*. Each "Monster" is a Dell 6300 PowerEdge server computer with four Intel 400 MHz Pentium II processors. The "Monsters" are interfaced with a SCRAMNet 150 megabit fiber optic ring network (see section A.3.2). An additional computer named *Janus* was also added to the network. *Janus* is a Dell Precision workstation with two Intel 450 MHz Pentium II processors. At this writing, the wake computation task has been converted to run simultaneously on the four processors of one "monster" and to interface with the helicopter model running on *Janus* or another "monster." Conversion to more extensive multiprocessing is pending.

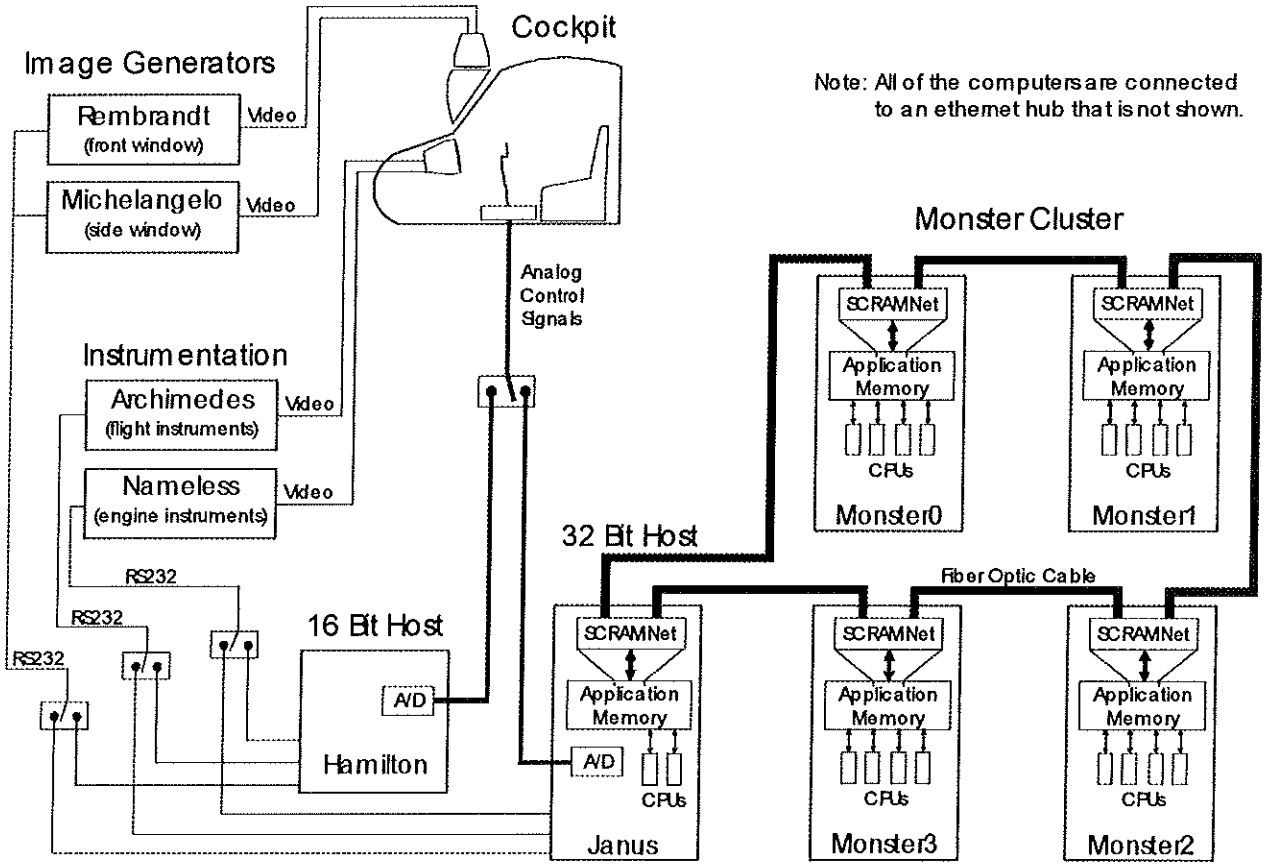


Figure 10: Simulator architecture.

A.3.1 MULTIPLE CPU UTILIZATION.

To benefit from multiple processors in the same computer, the simulation host programs are being restructured to use the multi-threading capabilities of the Windows NT 4 operating system. A computational thread represents a task that can be independently scheduled by the operating system and executed by a processor simultaneously with other threads being executed by other processors in the computer system. The threads of a single program all have access to the program's global data and data in the heap, but each thread has its own stack area for local variables and subroutine calls. At any instant, Windows NT will schedule one thread per processor, if the threads are available to be executed.

To take advantage of parallel processing, computations performed simultaneously on multiple processors, computations that can be performed independently must be identified. Care must be taken to insure that individual memory locations are not written to and read from simultaneously and that computations that depend on the results of previous calculations are executed in the proper order. Some tasks in the vortex lattice wake model lend themselves very well to parallel processing. For instance, the computation of the induced velocity at a lattice node is completely independent of the calculation for the induced velocity at the other nodes. The wake model tasks that can be computed in parallel have been identified and the wake program uses multiple threads to perform the calculations. Operation of the wake program using the four CPU's of one computer has been achieved.

Use of multiple threads does not automatically provide a performance increase by a factor equal to the number of processors to be utilized. A certain amount of overhead is incurred to manage the threads and to synchronize computations and access to shared resources. Preliminary results indicate that this overhead may be 20 to 30 percent. Additional investigation will be done to identify and reduce the overhead cost.

A.3.2 MULTIPLE COMPUTER UTILIZATION.

A SCRAMNet 150 megabit fiber optic ring network has been installed to interconnect the five computers to be used for the simulation host computations. SCRAMNet is a commercial product designed by the Systran Corporation with real-time simulation in mind. Each computer in a SCRAMNet network has a SCRAMNet expansion card installed. Each card has a certain amount of physical memory, RAM, installed – 1 megabyte for the FDL network. For each computer, this memory can be mapped into an application's address space as shared memory. Once the shared memory is mapped, write operations to the memory by one of the computers causes the SCRAMNet card to transmit the data to the memory on all of the other cards in the network. Programs running on the other computers can then read the new values from the shared memory. The networking aspect is transparent to the programs running on the different computers, making the SCRAMNet memory appear to be a block of shared memory that is accessed in the same manner as would be done if all of the programs were running on the same computer. Because this system dispenses with typical software networking protocols used to send data in packets, the network achieves extremely fast response and impressive throughput. Almost all networking overhead for the simulation program is eliminated.

The primary issues for dividing the computations across multiple computers using SCRAMNet are almost the same as for dividing the tasks among multiple CPUs on the same computer. Independent tasks must be identified in the same manner and a means of synchronizing the tasks must be devised just as for the multithreaded situation. A few additional problems arise because synchronization facilities are not available from the operating system. Alternative methods that will work in shared memory across multiple computers must be devised. Also, any data that is required by more than one computer must be placed into the SCRAMNet shared memory. This is not as much of an issue in multithreading within a single program, because all of the threads already have access to all of the memory.

At this time, neither the wake model program nor the helicopter/rotor program has been split to run on multiple computers across SCRAMNet. However, SCRAMNet has been utilized to run

the helicopter/rotor program on one computer and the wake program on a different computer in the network.

A.4 WAKE PROGRAM INTERFACE.

The rotor wake model is a self-contained program. The interface to the wake model is through SCRAMNet shared memory. A program with a rotor model that uses the wake model accesses it in three phases: a startup phase, an update phase that is performed in a loop, and a shutdown phase.

A.4.1 STARTUP PHASE.

During the startup phase, the rotor program must place certain data structures describing the rotor system into SCRAMNet memory. The first structure, called `WakeInterfaceData` by the wake program, contains parameters describing the rotor system as a whole. Only one of these structures is created in shared memory. The contents of this structure are detailed in table A-1. The second structure is called `WakeElementData` by the wake program. The wake program views the rotor system as if the rotor blades were modeled as a series of aerodynamic elements. A `WakeElementData` structure contains information about a single aerodynamic element. The contents of the structure are listed in table A-2. The rotor program must place one instance of the `WakeElementData` structure into shared memory for each aerodynamic element of each rotor blade. These structures must immediately follow the `WakeInterfaceData` structure in memory. They must be arranged in the same order as they are arranged along each rotor blade, beginning with each blade's root element closest to the hub and ending with that blade's tip element. The order of the blades is irrelevant. An array of structures, each with three double precision elements that represent the vector components of the inertial position of the tip of each blade, must also be placed in shared memory immediately following the `WakeElementData` structures. The order of the vector blade tip positions in the array must correspond to the blade order of the data used for the `WakeElementData` structures. The rotor code must also allocate two SCRAMNet interrupts that will be used as flags for synchronization between the two programs. One interrupt must be allocated in a sending mode and the other in a receiving mode.

Table A-1. WAKEINTERFACEDATA Structure Contents

Variable Name	Bytes, Integer /Float	Description
num_blades	4, I	number of rotor blades in the rotor system
num_elements_per_blade	4, I	number of aerodynamic elements per blade
rotor_radius	8, F	nominal radius of the rotor disc (ft)
air_density	8, F	density of the air (slug/ft^3)
time_step	8, F	time length of one wake step (sec)
Mode	4, F	program mode (OPERATE, RESET, etc.)
step_counter	4, I	number of wake passes
simulated_elapsed_time	8, F	elapsed time in the simulation (sec)
hub_position_inertial	3× 8, F	inertial position vector of the rotor hub (ft)
Inertial_to_azimuth	9× 8, F	inertial to azimuth passive rotation matrix
uniform_induced_velocity	3× 8, F	momentum theory induced velocity (ft/sec)

Table A-2. WAKEELEMENTDATA Structure Contents

Variable Name	Bytes, Integer/ Float	Description
Length	8, F	length of element (ft)
in_plane_air_speed	8, F	magnitude of the air relative velocity of the control point minus the spanwise component (ft/sec)
lift_per_unit_span	8, F	lift force per unit span at the control point (lb/ft)
Position	3× 8, F	inertial position of the root end of the element (ft)
control_point_position	3× 8, F	inertial position of the control point (ft)
control_point_air_velocity	3× 8, F	absolute velocity of the air at the control point (ft)

A C++ class, SCRAMNet, has been created to facilitate interactions with the SCRAMNet hardware, including allocation of memory and interrupts. It is possible to setup the structures in memory without the use of the SCRAMNet class, however, coordination between the two programs becomes more complicated. It will be assumed that the SCRAMNet class is to be employed. The following is an example of C++ code to allocate a block of data in SCRAMNet to be used as the shared memory interface between the rotor program and the wake program:

```
volatile void* scramnet_block_ptr =
  SCRAMNet::instance()
   .allocateMemory(sizeof(WakeInterfaceData) +
                   num_elements_per_blade * num_blades
                   * sizeof(WakeElementData) +
                   num_blades * sizeof(Vector),
                   wake_scramnet_block_name, true, 0);
```

The first parameter in this subroutine call represents the size of the memory block to be allocated, in bytes. In this example, num_blades is the number of rotor blades in the rotor system, and num_elements_per_blade is the number of aerodynamic elements per rotor blade. Vector is a data type that is comprised of three double precision floating point values. The second-to-last parameter is a boolean value indicating whether the block of data should be initialized before it is returned. The last parameter is the value with which the block should be filled if it is to be initialized. wake_scramnet_block_name is a variable of type string that

specifies the name of the block of data. This name is stored in shared memory with the data block and is used by the two programs to uniquely specify the block of SCRAMNet memory. When the wake program begins, it will wait until a block of memory with the correct name is allocated in shared memory, therefore, the same unique name must be used by both programs. The following code fragments can be used to allocate the SCRAMNet interrupts:

```
wake_start_pass_interrupt =
  SCRAMNet::instance()
   .allocateInterrupt(string("START:") +
                      wake_scramnet_block_name);

wake_pass_done_interrupt =
  SCRAMNet::instance()
   .allocateInterrupt(string("DONE:") +
                      wake_scramnet_block_name,
                      &wake_pass_done_event);
```

`wake_start_pass_interrupt` and `wake_pass_done_interrupt` are both variables of type `SCRAMNet::InterruptHandle`. The first call to allocate an interrupt returns a handle that can be used to trigger the SCRAMNet interrupt in the wake program. The parameter passed to the function is the name to be assigned to the interrupt. This name must be the same for both programs. Its value is the same as the name of the data block except that the string "START:" is prefixed. The second function call returns a handle to a SCRAMNet interrupt that will be signaled when the interrupt is triggered by the wake program. The name is the same as the block name except that the string "DONE:" is prefixed. The second parameter to this function call is the address of an object of type `Event`. The rotor program can wait for the `Event` object without consuming significant CPU time. This object is signaled when the interrupt is triggered.

In the setup phase, after the data block and the interrupts have been allocated, the rotor code must initialize the values stored in shared memory that remain constant during execution of the wake and rotor models. In the `WakeInterfaceData` structure, these values are the number of aerodynamic elements per blade, the nominal rotor disc radius, and the number of blades. Although the interface to the wake program has been written to allow the air density and the wake time step to change during simulation, these values are also currently initialized in the setup phase and remain unchanged during normal operation. The mode, an enumerated type, is also set in this phase to the value STARTUP (which currently has the numeric value 5). In the instances of the `WakeElementData` structures, the element lengths are assumed constant and, therefore, they must be initialized in the setup portion of the rotor program. It is important to note that the wake program cannot begin its initialization until this configuration data has been set by the rotor program. The value for the number of blades is used as a flag for the rotor program to signal the wake program that the appropriate data has been initialized in shared memory. Therefore, the variable for the number of blades must not be changed from zero to its actual value until all of the other configuration data has been set.

After initializing the shared memory data, the rotor program should wait on the event that the wake program uses to signal completion of a wake pass. Using the event from the example above, this could be accomplished using the following code:

```
wake_pass_done_event.waitForEvent(max_milliseconds_to_wait);
```

where `max_milliseconds_to_wait` is an integer value indicating an appropriate timeout value in milliseconds. The function returns true if the event is signaled or becomes signaled before the timeout expires. It returns false otherwise. The rotor program should reset the event to a non-signaled state after the wait function returns. This is accomplished in the following manner:

```
wake_pass_done_event.reset();
```

The wake program will trigger the SCRAMNet interrupt that sets this event when it completes its initialization and is ready to begin simulation looping. After this interrupt is received and the event is reset by the rotor program, the wake setup phase of the rotor program is complete.

A.4.3 UPDATE PHASE.

The update phase is typically performed by the rotor program while it is in a loop, simulating the rotor in operation. During the update phase, the rotor program must update any shared memory data that changes on a frame-by-frame basis except for the air velocity at the control point in the `WakeElementData` structure. After updating the data, the rotor program should signal the wake program that it is ready for the wake program to perform a pass. This is done by sending a SCRAMNet interrupt. Using the interrupt handle from the example above:

```
SCRAMNet::instance().sendInterrupt(wake_start_pass_interrupt);
```

After sending the interrupt, the rotor program should wait for, and then reset, the event that was created for the wake to signal a completed pass. This is the same as during the setup phase. After receiving the interrupt calling for the start of a pass, the wake program will update in shared memory the absolute velocity of the air in inertial coordinates at each blade element control point. After updating the data, the wake will send the interrupt indicating it has completed the pass. The data returned by the wake can be left in SCRAMNet memory and accessed by the rotor program as needed until the rotor program indicates that another pass should be performed by the wake, or it can be immediately copied to a more convenient location. The update phase should be performed in the same manner to accomplish a reset of the wake program. For reset, the mode variable should be set to RESET (4), as opposed to OPERATE (1) which is the setting while the rotor program is in the simulation loop.

A.4.4 SHUTDOWN PHASE.

The shutdown phase exists merely to alert the wake program that the rotor program is either exiting or no longer requires the services of the wake program. The rotor program should set the mode variable in the `WakeInterfaceData` structure to the value `SHUTDOWN` (6) and then send the SCRAMNet interrupt that signals that the wake should begin a pass. When the wake program receives the interrupt, the wake program terminates.

Acknowledgments

This work was supported by the Federal Aviation Administration under Cooperative Agreement 98-G-018, Task E.

Kenneth Graham was also supported by a doctoral fellowship awarded by The Link Foundation.

References

1. Katz, A., "Dynamic Simulation Models for the UH1-H and TH-67 Helicopters," UA FDL report 97S04A, Prepared for Anacapa Sciences, Inc. under Subcontract ASI-97-DHH-1058-13 from US Army Research Institute, Ft. Rucker, AL., October 30, 1997.
2. Graham, K., and Katz, A., "A Blade Element Model in a Low Cost Helicopter Simulator," AIAA Paper 94-3436-CP, Proceedings of the AIAA Flight Simulation Technologies Conference, Scottsdale, Arizona, August 1-3, 1994, pp 287-293.
3. Katz, A. and Graham, K. "Control Issues for Rigid-in-Plane Helicopter Rotors," Journal of Aircraft vol. 33, #2, pp. 311-315, March-April 1996.
4. Katz, A., "Residual Shaft Bending by Helicopter Rotors," Journal of Aircraft vol. 34, #5, pp. 688-690, September-October 1997.
5. Katz, A. and Waits, T. "Global Recursive Dynamics of Articulated Tree Structures," Computational Mechanics vol. 23 #4 pp. 288-298, May 1999
6. A. Katz, J. Van Pelt, T. Waits and W. Liu, "Simulator Construction Set (SCS) Phase 1", UA FDL report 95S01B prepared for Loral Western Development Labs, San Jose, CA under Subcontract SO-242825-A, item 05, January 8, 1996.
7. Rosen, A. and Isser, A. "A New Model of Rotor Dynamics During Pitch and Roll of a Hovering Helicopter," Journal of the American Helicopter Society vol. 40, #3, pp.17-28, July 1995.
8. Rosen, A. and Isser, A. "A Model of the Unsteady Aerodynamics of a Hovering Helicopter Rotor that includes Variations of the Wake Geometry," Journal of the American Helicopter Society vol. 40, #3, pp. 6-16, July 1995.
9. Prouty, R. W., "The Case of the Cross-Coupling Mystery," Rotor & Wing, June 1994 pp. 48-49.